# ngspice Linear Solvers — KLU vs. Sparse 1.3

Behavior, defaults, and limitations across DC / AC / transient / noise

Ngspice + OpenVAF Enhancements project

July 2026

## Contents



## ngspice linear solvers — KLU vs. Sparse 1.3 (behavior, defaults, limitations)

This build of ngspice-46 ships two direct linear solvers, and they are not fully interchangeable across analysis types. This note records exactly how they differ — which is the default, how to select and confirm each, and where KLU falls short — based on a direct read of the source and a solver-by-solver sweep of the whole `examples/` suite.

### TL;DR

| | KLU | Sparse 1.3 |
|---|---|---|
| Availability in this build | Compiled in (SuiteSparse, statically linked — 44 `klu_*` symbols in the binary) | Always present (ngspice's own solver) |
| Default? | No | Yes — this build defaults to Sparse 1.3 |
| How to select | `.option klu` | `.option sparse` (or just the default) |
| DC op / DC sweep | ✓ correct (one caveat below) | ✓ correct |
| AC | ✓ correct | ✓ correct |
| Transient | ✓ correct (one caveat below) | ✓ correct |
| Noise (**`.noise`**) | ✓ correct (since E-113) | ✓ correct |
| Pole-zero (**`.pz`**) | ✓ single-ended; ⚠balanced-output Sparse-only | ✓ correct |
| Sensitivity (**`.sens`**, DC & AC) | ✓ correct (since E-114) | ✓ correct |
| Distortion (**`.disto`**) | ✓ correct (since E-115) | ✓ correct |

Practical guidance: leave the default (Sparse 1.3) unless you have a specific reason to switch. Sparse 1.3 runs every analysis in the suite. Since Enhancement-113 KLU also runs noise and single-ended pole-zero correctly, and since Enhancement-114 it runs DC/AC sensitivity, and since Enhancement-115 distortion (**`.disto`**) correctly; the only analysis still Sparse-only under KLU is balanced-output pole-zero. Reach for `.option klu` on large, sparse DC/AC problems where KLU's ordering and factorization are faster, and expect it to be less robust on stiff transient edges and degenerate topologies (see the two genuine discrepancies below).

### Selecting and confirming the solver

The netlist option is a simple flag:

```
.option klu       * use KLU (SuiteSparse)
.option sparse    * use the legacy Sparse 1.3  (this is also the default)
```

Because unknown `.option` keywords are silently ignored by ngspice, "it ran without complaint" is *not* evidence the solver changed. To confirm which solver is actually active, use the interactive `option` command, which prints the live `CKTkluMODE`:

```
.control
run
option        * prints, among other things:  "Matrix solver:  KLU"  or  "Sparse 1.3"
.endc
```

On the committed `bin/` binary this reports **`Sparse 1.3`** for a bare deck and for `.option sparse`, and **`KLU`** only when `.option klu` is given — confirming Sparse 1.3 is the default here.

## Noise and pole-zero under KLU (Enhancement-113)

ngspice used to refuse both outright (`noisean.c` / `pzan.c` printed "not (yet) supported with 'option KLU'"). The real reason for noise was subtler than "not wired up": noise uses the adjoint method — it solves the *transposed* system $A^T \cdot x = e$ via `SMPcaSolve` — and `SMPcaSolve`'s KLU branch was calling the non-transposed `klu_z_solve` where its Sparse branch calls `spSolveTransposed`. That silently produced the wrong noise for any asymmetric matrix (every circuit with a transistor or controlled source), which is why it was disabled rather than merely slow.

Enhancement-113 fixes the adjoint solve (`klu_z_tsolve`) and lifts the guards, so under KLU:

- Noise is correct — verified identical to Sparse on resistor thermal noise, asymmetric VCCS circuits, OSDI device models, and integrated totals. (`.sp` S-parameters share the same adjoint solve and are corrected too.)
- Single-ended pole-zero (grounded output reference) runs correctly.
- Balanced-output pole-zero (non-grounded reference) stays Sparse-only: its zeros phase folds columns at solve time, which Sparse survives via dynamic Markowitz re-ordering but KLU's fixed symbolic factorization cannot; a targeted guard now directs that case to `.option sparse`.

Enhancement-114 then fixes sensitivity under KLU. Sensitivity builds an auxiliary perturbation matrix `delta_Y` that is a plain Sparse matrix (it is only multiplied, never factored); two KLU setup blocks were gated on the *main* matrix's flag and dereferenced the NULL `delta_Y->SMPkluMatrix`, segfaulting on every DC/AC `.sens` deck. Gating them on `delta_Y`'s own flag keeps it Sparse under KLU too, so both DC and AC sensitivity now match Sparse exactly.

- Distortion (**`.disto`**) was Sparse-only — `distoan.c` had no KLU code, so under `.option klu` the (complex) distortion solves ran against a matrix left in real mode and silently produced no output. Surfaced while validating E-114. Enhancement-115 fixes it: it converts the KLU matrix real↔complex around the distortion solve loop (exactly as `acan.c` does for AC), so `.disto` now matches Sparse bit-for-bit.

## Three genuine KLU discrepancies

Beyond the noise/pole-zero refusals, three examples in the suite produce a different (wrong) result under KLU than under Sparse. All are marked `KLU_XFAIL` in the dual-solver harness so they are exercised and tracked rather than silently skipped.

1. opamp741 — stiff transient diverges. The transistor-level opamp741 example (a $\mu$A741 from ~70 lines of Verilog-A BJT):

- Sparse 1.3: the large-signal slew test (`pulse(-5 5 …)` into the follower) runs the full `tran 20n 80u` cleanly (~4058 points), slew rate $\approx 0.54$ V/$\mu$s.
- KLU: the same run diverges at the slewing edge — `v(out)` overshoots past the $-5$ V rail to $\approx -6.16$ V and ngspice aborts the timestep at t $\approx 2.03$ $\mu$s (~133 points), so the characterization can't complete.

A convergence/timestep robustness difference on a stiff circuit: Sparse's pivoting survives the fast edge and KLU's does not. (E-111's line search does not help — it damps the DC-op Newton, not per-timepoint transient iterations.)

2. groundcontrib — wrong DC answer on a degenerate topology. The groundcontrib example drives a single node `p` with a node-to-ground voltage contribution `V(p) <+ 1.5` through a load resistor to ground. On this exact same deck, changing only the solver:

```
.option sparse  ->  v(p) = 1.5   (correct)
.option klu     ->  v(p) = 0.0   (wrong)
```

This is a genuine KLU correctness miss (not a refusal or a divergence): on this degenerate single-node system KLU returns zero. It is the reason to prefer Sparse for unusual/degenerate one-node topologies. The ~90 other OSDI examples run correctly under KLU, so the issue is specific to this topology, not OSDI generally.

3. hierbranch — hierarchical branch *current* probes read zero. The hierbranch example probes currents on hierarchical/synthesized ammeter branches (the E-86 zero-volt sources). On the same deck, changing only the solver:

- Sparse 1.3: all six checks pass — node voltages *and* branch currents (`I(top.d1.branch(<p>)) = 5 mA`, `i(V1) = −10 mA`).
- KLU: the node voltages are correct (`V(top.a1.b) = 1.34 V`, `V(top.d1.branch(va,vb)) = 2.5 V`), but the branch currents read `0`.

Deterministic (KLU always 4/6, Sparse always 6/6), and independent of the E-114 sensitivity fix — the DC KLU solve is untouched by E-114. It was previously masked: the old deck-injector left a stale `.option sparse` in the deck that had never been restored, so the "klu" child silently ran Sparse and reported a false pass. Fixing that injector-restore bug (E-114) exposed the real KLU miss on the synthesized current-unknown rows — a candidate for a future enhancement.

## The dual-solver test harness

Every `verify_*.py` in `examples/` is run under both solvers automatically. Each script calls `check_both_solvers(__file__)` (right after importing `_setup`); on a normal run that re-executes the script once per solver — injecting `.option sparse` / `.option klu` into every ngspice deck — and prints a combined verdict:

```
=== BOTH-SOLVER RESULT [ceil]: sparse=PASS  klu=PASS => OK ===
```

KLU's one remaining unsupported analysis (balanced-output pole-zero) is auto-detected and reported `SKIP`; the three `KLU_XFAIL` examples above are expected-fail under KLU. Env escape hatches: `NGSPICE_SOLVER=klu|sparse` runs once under one solver; `NG_BOTH=0` disables the dual run.

Sweep result (all 101 machine-checkable scripts): 101/101 OK — every example is `sparse=PASS` with `klu` in `{PASS, SKIP, XFAIL}`, i.e. the two solvers agree wherever KLU is applicable. (The `linesearch` example initially *crashed* under KLU, which Enhancement-112 fixed; Enhancement-113 then moved 10 examples from KLU-skipped to KLU-passing by enabling noise and single-ended pole-zero; Enhancement-114 fixed the AC/DC-sensitivity crash under KLU, and — by fixing the deck-injector restore bug — un-masked a third `XFAIL`; Enhancement-115 then fixed distortion, so the `analyses` example now runs fully under KLU. The `XFAIL` entries are opamp741, groundcontrib, and hierbranch.)

Conclusion: for DC, AC, transient, noise, single-ended pole-zero, sensitivity, and distortion the two solvers agree across the suite, with exactly three KLU exceptions (the stiff opamp741 transient, the degenerate groundcontrib DC topology, and hierbranch's hierarchical branch-current probes), while balanced-output pole-zero is Sparse-1.3-only by ngspice design. Sparse 1.3, the default, covers everything — which is why it is the default and why the dual-solver harness keys correctness off it.